



Mansoura University
Faculty of Computers and Information



Operating Systems

File-System Interface

Dr. Mohamed Handosa

Assistant Professor of Computer Science

handosa@mans.edu.eg

File Concept

- A file is stored on secondary storage.
- It is a named collection of related data.
- It represents a contiguous logical address space.
- File content and type are defined by the creator.
- Common file types include:
 - **Data files:** numeric, character, binary
 - **Program files:** compiled code (e.g., executable files)

File Attributes

- Every file has metadata associated with it:
 - **Size:** current file length
 - **Type:** defines file behavior
 - **Name:** human-readable identifier
 - **Identifier:** unique tag for system use
 - **Location:** pointer to file's physical location
 - **Protection:** access permissions (read, write, execute)
 - **Timestamps and user ID:** for security, backup, and auditing
- These attributes are maintained in the directory structure on the disk.

File Operations

- Files support a range of standard operations:
 - **Create**: generate a new file entry
 - **Write**: place data at current write pointer
 - **Read**: fetch data at current read pointer
 - **Seek**: reposition within a file
 - **Delete**: remove a file
 - **Truncate**: erase file content but retain structure
 - **Open**: load file attributes from disk to memory
 - **Close**: write memory-based metadata back to disk

Open Files

- To manage open files, the OS maintains:
 - **Open-file table:** to track all open files
 - **File pointers:** per-process indicators of read/write positions
 - **Open count:** how many processes have opened the file
 - **Location info:** helps access file quickly
 - **Access rights:** stored per process

File Locking

- Prevents concurrent access.
- **Exclusive lock:** allows one writer
- **Shared lock:** allows multiple readers
- **Locking types:**
 - **Mandatory:** enforced by OS
 - **Advisory:** cooperation between processes
- **Java Example:**
 - Use of **FileLock** in Java via **RandomAccessFile** and **FileChannel**.
 - This Allows developers to lock parts of a file for reading or writing.

File Types and Extensions

- File types are often recognized by extensions (e.g., .txt, .exe).
- OS and applications use these to associate files with programs.

File Structure

- File organization methods:
 - **Unstructured:** byte or word sequences
 - **Simple structure:** lines of fixed or variable length
 - **Complex structure:** e.g., formatted documents or relocatable executables
- File structure can be imposed by:
 - Operating system
 - Application programs

Access Methods

- **Sequential access**
 - Reset pointer to start
 - Read/write data in order
- **Direct access**
 - Block number used as index
 - Allows reading or writing to any block (n)
- **Indexed access**
 - File has an index to quickly locate data
 - Multi-level index used when data is large
 - Example: IBM's ISAM or VMS index/relative files

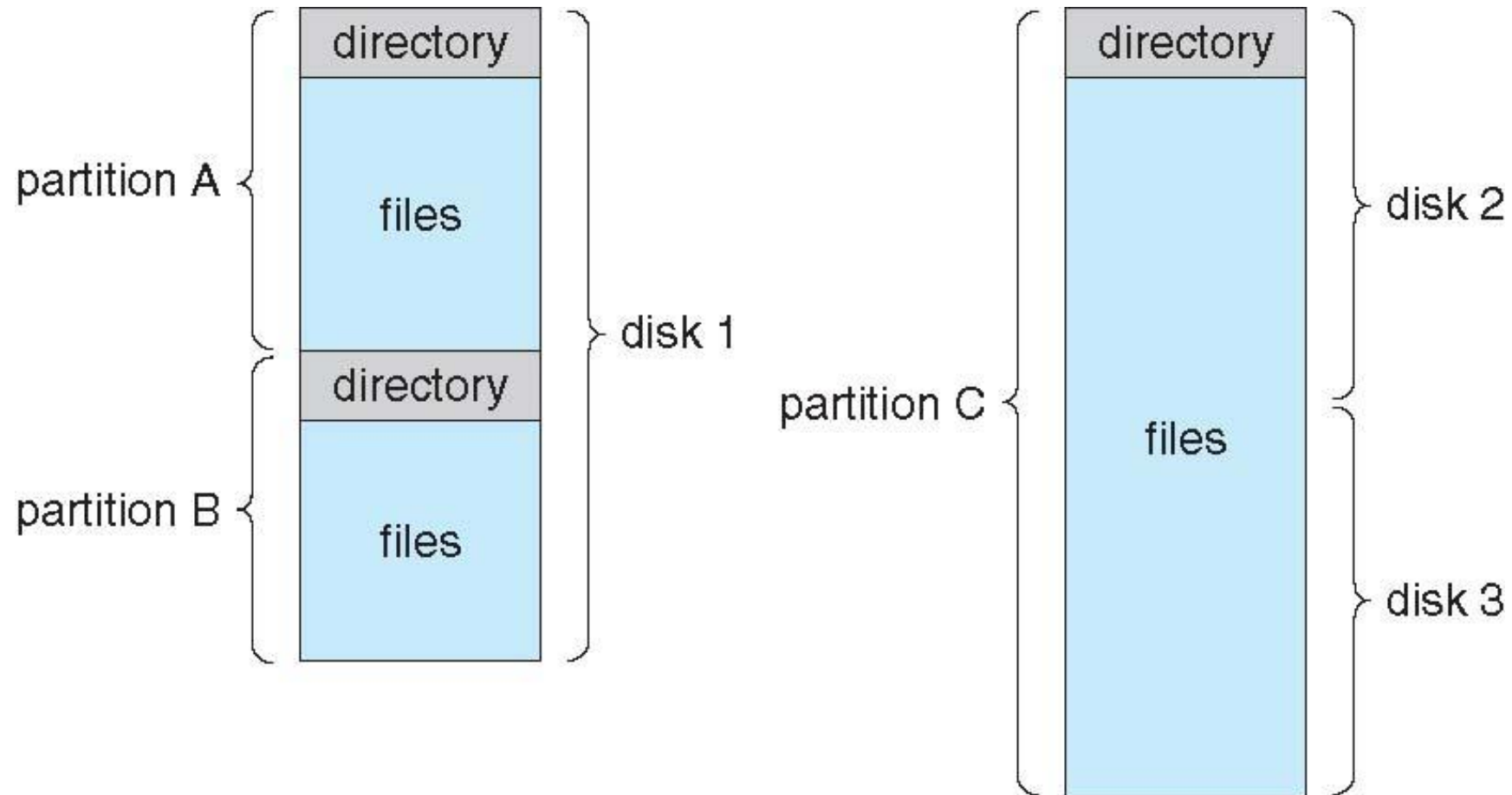
Directory Structure

- A **directory** is a collection of file metadata.
- Both files and directories are stored on disk.
- Directories help:
 - Locate files efficiently
 - Organize and manage files

Disk Structure and File Systems

- Disks may be divided into partitions.
- A **partition** (or **volume**) holds a file system.
- It tracks info in a **volume table of contents (VTOC)**.
- Each partition may be:
 - Raw (no file system)
 - Formatted with a file system
 - Using **RAID** to achieve redundancy

A Typical File-system Organization



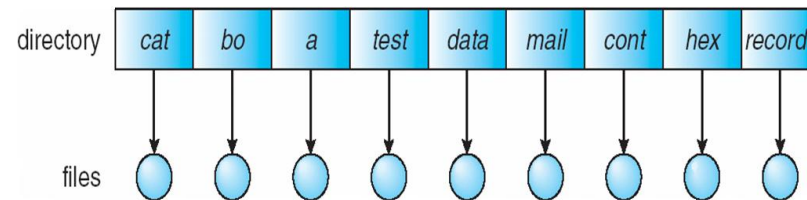
Directory Operations

- List
- Create
- Delete
- Search
- Rename files
- Traverse file system hierarchies

Directory Organization Schemes

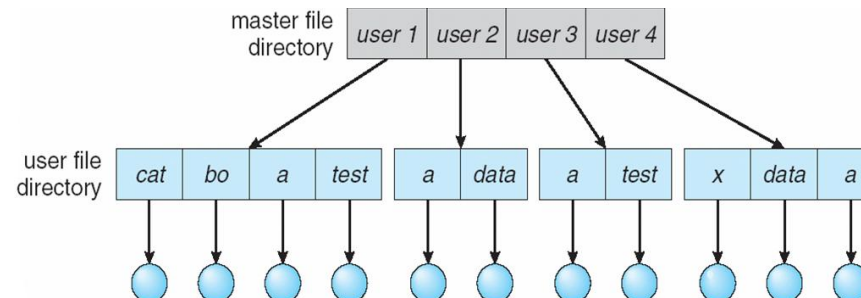
- **Single-Level Directory**

- All files in one folder (naming and grouping problems)



- **Two-Level Directory**

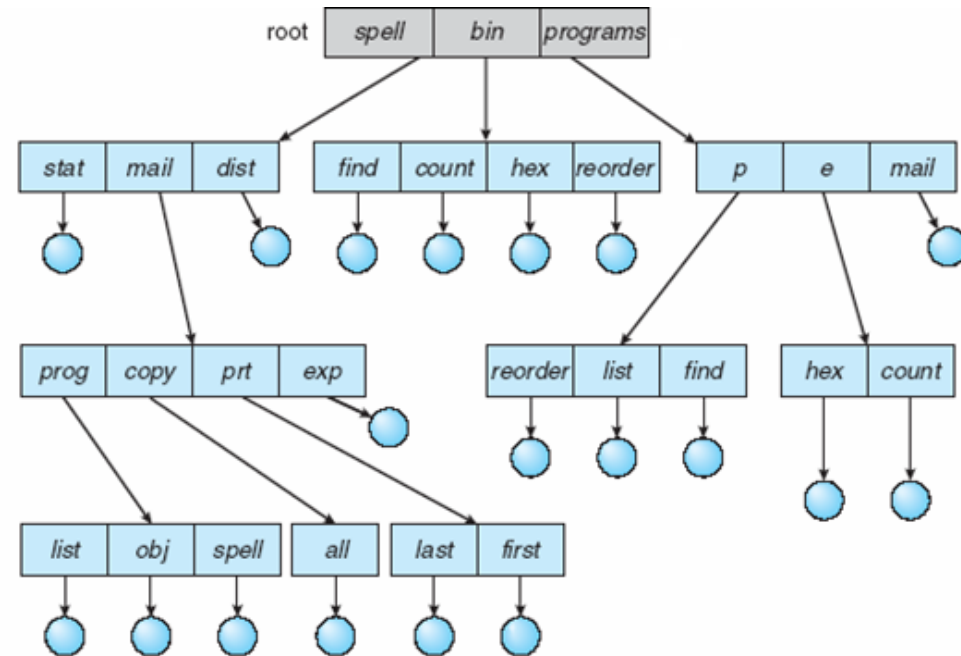
- Separate directory for each user (better isolation between users)



Directory Organization Schemes (Cont.)

- **Tree-Structured Directory**

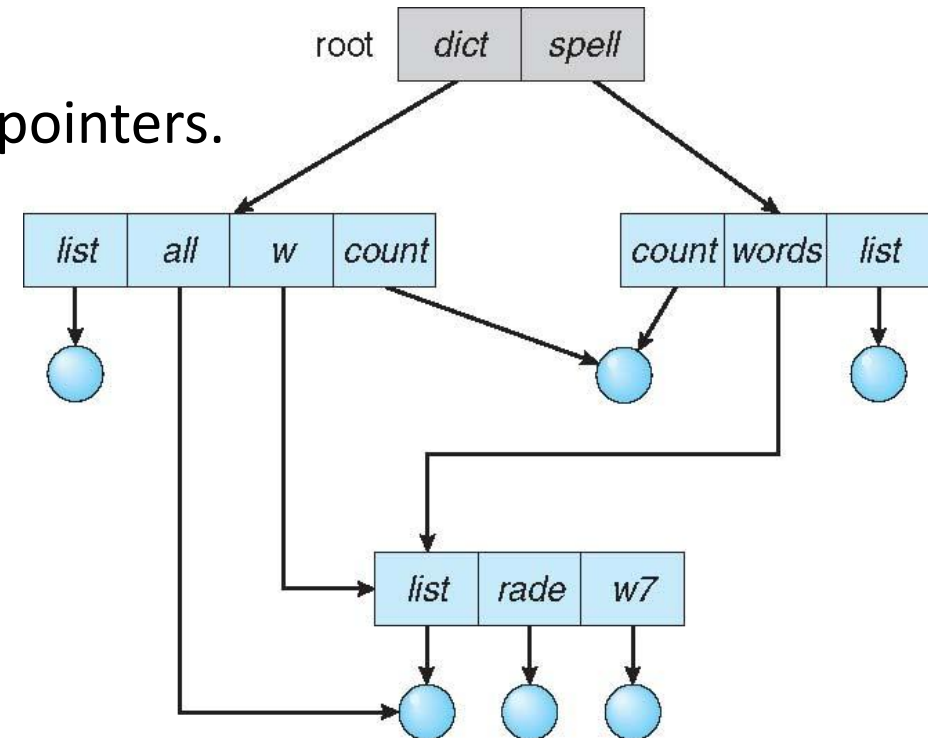
- Allows hierarchy
- Easy grouping and navigation
- Supports absolute and relative paths



Directory Organization Schemes (Cont.)

- **Acyclic-Graph Directory**

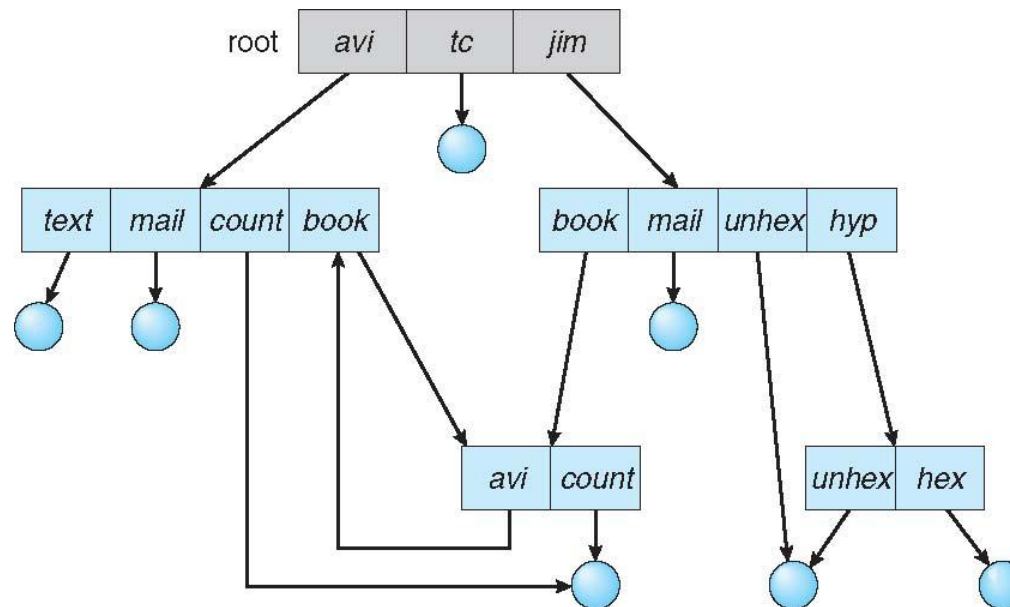
- Allows shared files and directories.
- Uses links (aliases) to avoid duplication.
- Careful handling needed to avoid dangling pointers.



Directory Organization Schemes (Cont.)

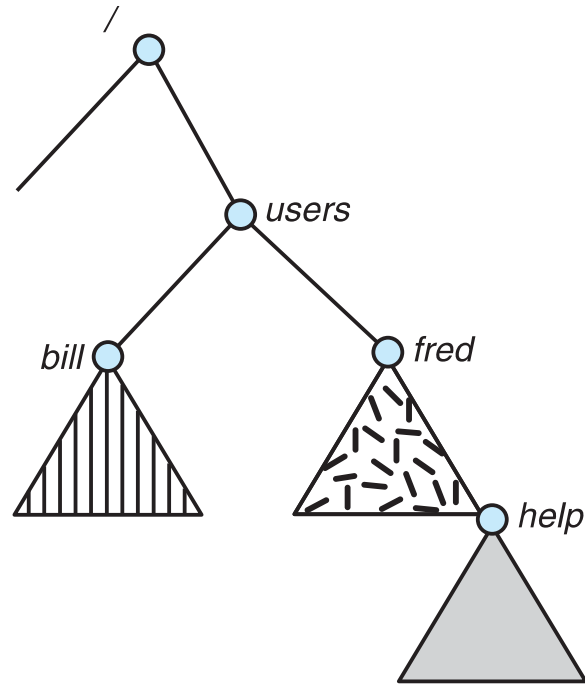
- **General Graph Directory**

- Supports links between any nodes (risk of cycles).
- Requires garbage collection as well as cycle detection.

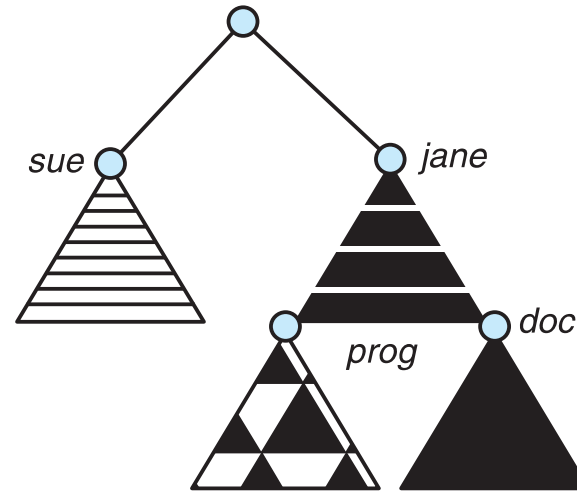


File-System Mounting

- A file system must be mounted before it can be accessed.



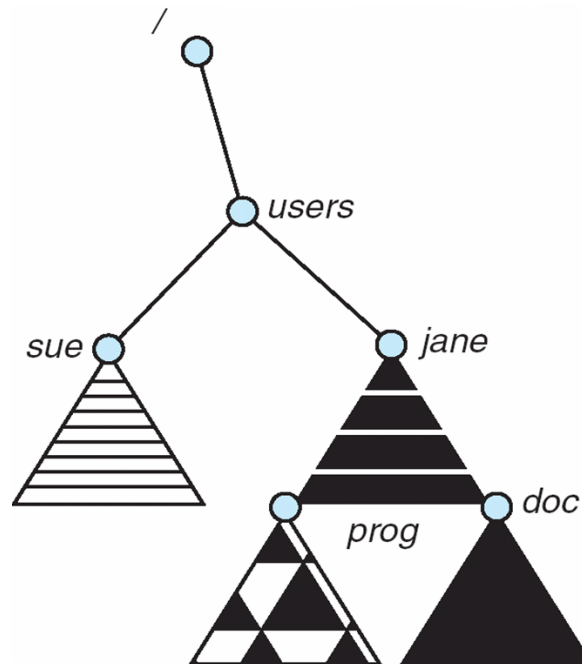
(a)



(b)

File-System Mount Point

- A location in existing directory structure.
- Defines where the new file system is attached.



File Sharing

- On **multi-user systems**, files can be shared:
 - Via **user ID and group ID** for access control
 - Over **networks** using file sharing protocols like:
 - **NFS** (Unix)
 - **CIFS** (Windows)
- **Distributed file systems** allow mounting remote systems.
- **LDAP, DNS, and Active Directory** manage remote access info.

File Sharing – Failure Modes

- Issues include:
 - Directory corruption
 - Network/server failures
- Recovery strategies:
 - Use of **stateless protocols** like NFS v3 (each request is independent)

File Sharing – Consistency Semantics

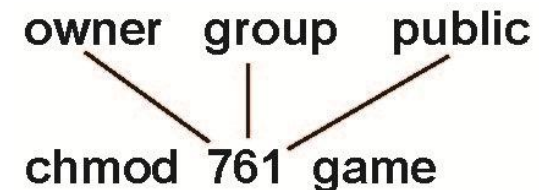
- Define how updates to files are seen by other users:
 - **Unix**: updates visible immediately
 - **AFS**: changes visible after file is closed (session semantics)
- Depends on system's sync model and network latency

Protection

- File **owners** define permissions.
- Types of access: **read, write, execute, append, delete, list**
- Protection mechanisms ensure only authorized access is granted.

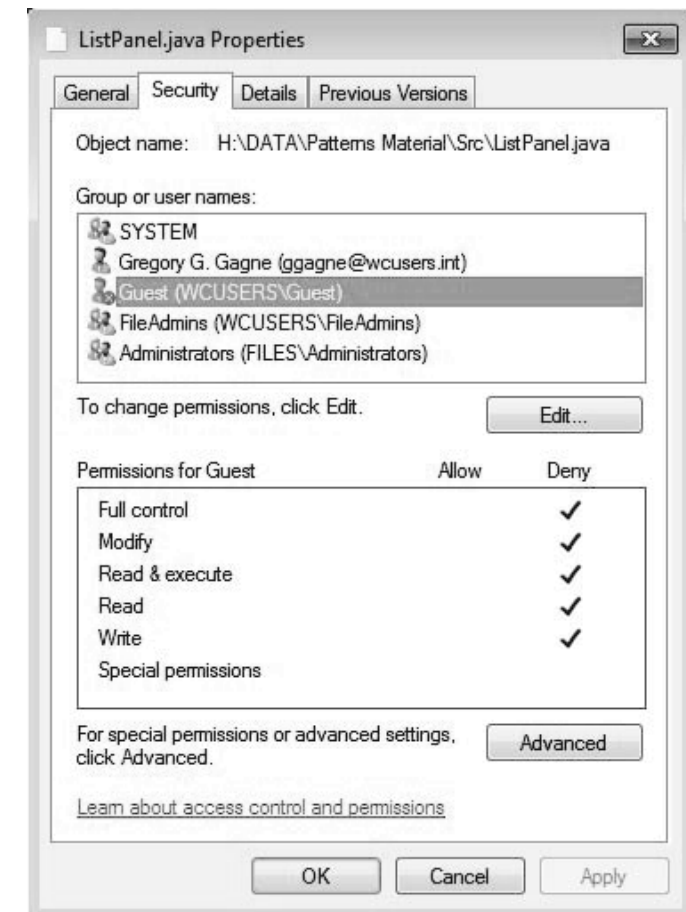
Access Lists and Groups (Unix/Linux Example)

- **Access mode:** read, write, execute.
- Permissions represented as 3-digit binary (RWX).
- There are three classes of users (owner – group – public).
- There are two common command to manage permissions:
 - The command **chmod** can change file permissions
 - Example: **chmod 761 game**, where game is a file name
 - Owner → 7 (111 = RWX) group → 6 (110 = RW-) public → 1 (001 = --X)
 - The command **chgrp** can change file group ownership
 - Example: **chgrp [groupname] [file]**



GUI-Based Access Control (e.g., Windows)

- Graphical systems allow file permissions to be set through access control lists (ACLs).
- Permissions include **read, write, modify**, etc.



Thank You